

Simple API with JWT Authentication and RBAC

--- main.py ---

```
from fastapi import FastAPI
from auth import router as auth_router
from crud import router as project_router

app = FastAPI()

app.include_router(auth_router, prefix="/auth")
app.include_router(project_router, prefix="/projects")
```

--- models.py ---

```
from sqlmodel import SQLModel, Field
from typing import Optional

class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    username: str = Field(index=True, unique=True)
    hashed_password: str
    role: str

class Project(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    description: str
    owner_id: int
```

--- schemas.py ---

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    username: str
    password: str
    role: str

class UserLogin(BaseModel):
    username: str
    password: str

class ProjectCreate(BaseModel):
    name: str
    description: str
```

--- auth.py ---

```
from fastapi import APIRouter, Depends, HTTPException
from jose import jwt
from passlib.context import CryptContext
```

```

from datetime import datetime, timedelta
from models import User
from database import session
from schemas import UserCreate, UserLogin

SECRET_KEY = "secret"
ALGORITHM = "HS256"
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
router = APIRouter()

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password):
    return pwd_context.hash(password)

@router.post("/register")
def register(user: UserCreate):
    hashed_pw = get_password_hash(user.password)
    db_user = User(username=user.username, hashed_password=hashed_pw, role=user.role)
    session.add(db_user)
    session.commit()
    return {"msg": "User registered"}

@router.post("/login")
def login(user: UserLogin):
    db_user = session.query(User).filter(User.username == user.username).first()
    if not db_user or not verify_password(user.password, db_user.hashed_password):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    token_data = {"sub": db_user.username, "role": db_user.role}
    token = jwt.encode(token_data, SECRET_KEY, algorithm=ALGORITHM)
    return {"access_token": token}

```

--- crud.py ---

```

from fastapi import APIRouter, Depends, HTTPException
from models import Project, User
from schemas import ProjectCreate
from database import session
from jose import jwt
from fastapi.security import OAuth2PasswordBearer

router = APIRouter()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
SECRET_KEY = "secret"
ALGORITHM = "HS256"

def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    user = session.query(User).filter(User.username == payload["sub"]).first()
    return user

@router.get("/")
def get_projects(user=Depends(get_current_user)):

```

```
        return session.query(Project).all()

@router.post("/")
def create_project(project: ProjectCreate, user=Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    db_project = Project(**project.dict(), owner_id=user.id)
    session.add(db_project)
    session.commit()
    return db_project
```

--- database.py ---

```
from sqlalchemy import create_engine, SQLModel, Session

DATABASE_URL = "postgresql://user:password@localhost/dbname"

engine = create_engine(DATABASE_URL, echo=True)
SQLModel.metadata.create_all(engine)
session = Session(engine)
```

--- requirements.txt ---

```
fastapi
sqlmodel
uvicorn
psycopg2-binary
passlib[bcrypt]
python-jose
```